Announcement:

Make-up class for Lunar New Year day:
Zoom recording on 21 Feb., 2026, Sat,
from 9:30am to 12:20pm
Zoom Meeting Link:
https://ntu-sg.zoom.us/j/87272699261?pwd=RbZOtilqgCboz4rrVkL8hr9pzPegyx.1
Meeting ID: 872 7269 9261
Passcode: 712179

EE6406: Analytic and Ensemble Learning

Lecture 6

Advanced Analytic Classification

Overview

1. Introduction to Classification Error Based Learning
2. Learning Based on Classification Total Error Rate
3. Learning Based on Classification Operating Characteristic
4. A Generalized Learning Framework
   - Data Transformation
   - Between TAUC and AUC Solutions
   - Between TAUC and FLD Solutions
   - Between TER and LSE Solutions
   - Relationship with Bayesian Inference
   - Summary of Classifier Relationships

Introduction

- Apart from the error magnitude based methods covered in previous lecture, a more matching approach for classification is to directly formulate the classification metric as the learning objective.
- This approach has been a classical subject of study known as *classification error* (CE) based methods. However, due to the counting nature of the classification metric (e.g., by accumulating the number of successful prediction counts), attempts to the problem had inevitably resulted in solving a nonlinear formulation.
- In this lecture, we introduce two classifier learning solutions which can be obtained in closed form and yet possesses nonlinear mapping capability. The first method directly uses the total error rate (TER) as the learning objective function while the second method uses uses area under the ROC curve (AUC) as the learning objective function.

Total Error Rate Based Learning

The total error rate (TER = FPR + FNR) would be a suitable candidate for objective function formulation since it is directly accumulating the classification error count in normalized form. Let's denote the positive and negative examples of variables for $\boldsymbol{x}$ and m by the superscripts + and −, respectively, and by expressing in terms of classification based on a threshold, τ, the TER can be written as:

$$\begin{aligned} \text{TER} &= \text{FPR} + \text{FNR} \\ &= \frac{1}{m^-}\sum_{j=1}^{m^-} cls(g(\boldsymbol{x}_j^-, \boldsymbol{w}) \geqslant \tau) + \frac{1}{m^+}\sum_{i=1}^{m^+} cls(g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau), \end{aligned} \quad (5.65)$$

where $cls(\cdot)$ is the classification function given in (5.68) which is defined for a particular threshold τ. In other words, $cls(g(\boldsymbol{x}_j^-, \boldsymbol{w}) \geqslant \tau) = 1$ whenever $g(\boldsymbol{x}_j^-, \boldsymbol{w}) \geqslant \tau$ holds true and $cls(g(\boldsymbol{x}_j^-, \boldsymbol{w}) \geqslant \tau) = 0$ otherwise. Similarly, $cls(g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau) = 1$ whenever $g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau$ holds true and $cls(g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau) = 0$ when $g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau$ is not true.

Total Error Rate Based Learning

By noting the symmetry between $cls(g(\boldsymbol{x}_j^-, \boldsymbol{w}) \geqslant \tau)$ and $cls(g(\boldsymbol{x}_i^+, \boldsymbol{w}) < \tau)$, a simple change of variables can unify the two classification functions $cls(g^- \geqslant \tau)$ and $cls(g^+ < \tau)$ into one loss form. Define $\epsilon_j = g(\boldsymbol{x}_j^-, \boldsymbol{w}) - \tau$ for $j = 1, 2, \ldots, m^-$ and $\varepsilon_i = \tau - g(\boldsymbol{x}_i^+, \boldsymbol{w}) - \Delta$ for $i = 1, 2, \ldots, m^+$. Here, $\Delta \to 0$ which is to account for the strict inequality in FNR of (5.65) can be ignored in practice. Then the TER loss function can be expressed in terms of ϵ_j and ε_i which has a threshold at zero, giving

$$\text{TER} = \frac{1}{m^-} \sum_{j=1}^{m^-} L(\epsilon_j \geqslant 0) + \frac{1}{m^+} \sum_{i=1}^{m^+} L(\varepsilon_i \geqslant 0). \tag{5.66}$$

Total Error Rate Based Learning

The goal to train a predictor as a classifier is then to find the minimizer of TER given by

$$\begin{aligned} \hat{\boldsymbol{w}} &= \arg\min_{\boldsymbol{w}} \mathrm{TER} \\ &= \arg\min_{\boldsymbol{w}} \left\{ \frac{1}{m^-} \sum_{j=1}^{m^-} L(\epsilon_j \geqslant 0) + \frac{1}{m^+} \sum_{i=1}^{m^+} L(\varepsilon_i \geqslant 0) \right\}. \end{aligned} \tag{5.67}$$

Here, loss function $L(\cdot)$ can be considered an alternative version of the classification decision function (5.68) with positive and negative targets (instead of 0 and 1) thresholding at $\tau = 0$.

$$cls(g(\boldsymbol{x}_j)) = \begin{cases} 1 & \text{if} \quad g(\boldsymbol{x}_j) \geqslant \tau \\ 0 & \text{if} \quad g(\boldsymbol{x}_j) < \tau \end{cases}, \quad j = 1, 2, \ldots, n. \tag{5.68}$$

Total Error Rate Based Learning

The formulation in (5.67) is not a differentiable function where conventional optimization techniques can apply. A natural way to solve this problem is to approximate the zero-one step loss function by a smooth *sigmoid* function given by

$$\sigma(\theta) = \frac{1}{1 + e^{-\gamma\theta}}, \quad \gamma > 0 \tag{5.69}$$

where $\theta \in \mathbb{R}$ is an input variable (such as ϵ_j and ε_i in (5.67)) and $\gamma \in \mathbb{R}$ controls the slope of the activation (see Fig. 1).
The minimization problem then becomes a smooth and differentiable one given by:

$$\min_{\boldsymbol{w}} \text{TER} \approx \min_{\boldsymbol{w}} \left\{ \frac{1}{m^-} \sum_{j=1}^{m^-} \sigma(\epsilon_j \geqslant 0) + \frac{1}{m^+} \sum_{i=1}^{m^+} \sigma(\varepsilon_i \geqslant 0) \right\}. \tag{5.70}$$

Total Error Rate Based Learning

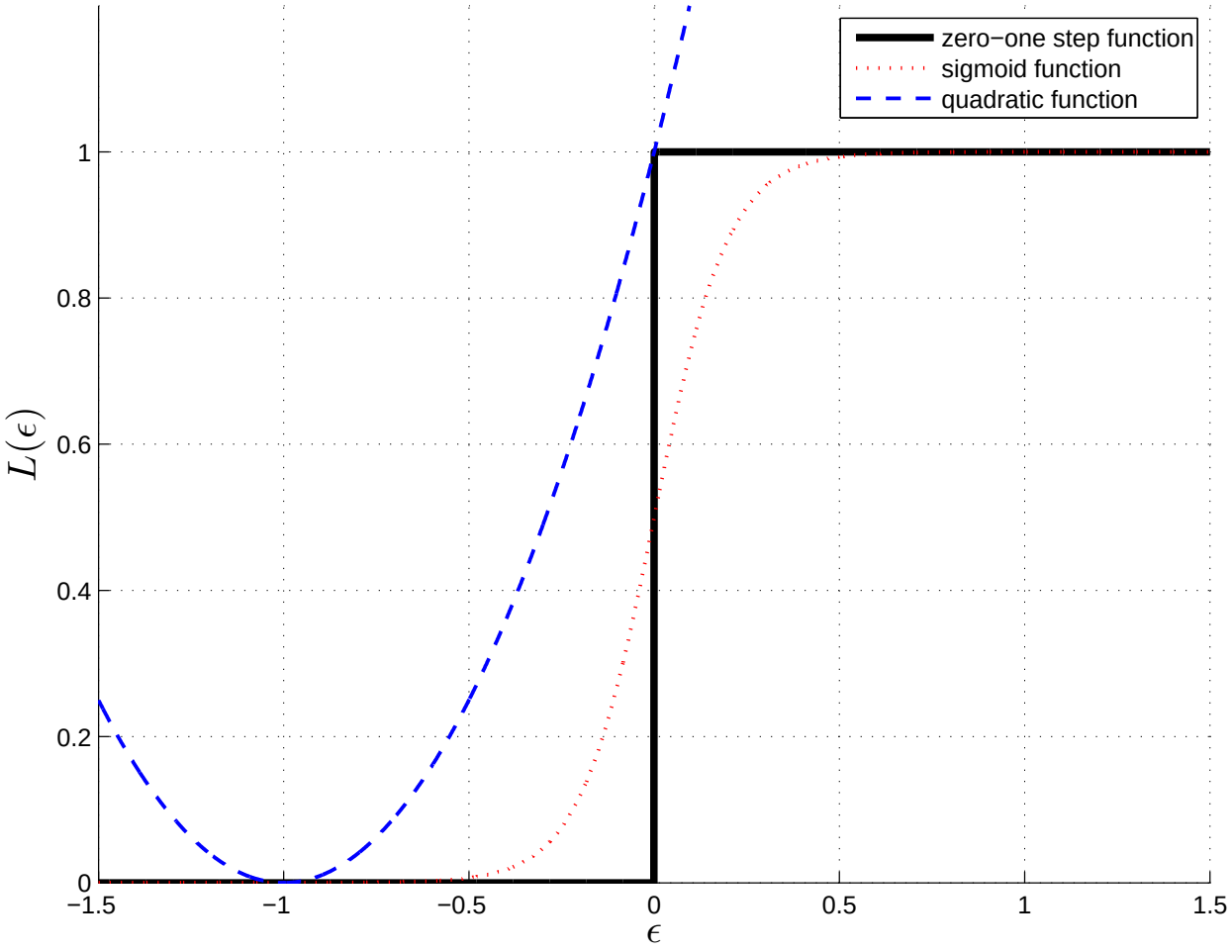


Figure 1: Sigmoidal and quadratic approximations to the zero-one step loss function. The sigmoidal function is adopted as $1/(1+\exp(-10\epsilon))$

Total Error Rate Based Learning

- However, such a sigmoidal approximation is often associated with problems related to solution search.
- The first problem is the resulted nonlinear formulation with respect to the learning parameter vector $\boldsymbol{w}$ (i.e., $\sum \sigma(\boldsymbol{w})$, recall that ϵ in $\sigma(\epsilon \geqslant 0)$ is a function of $\boldsymbol{w}$). This in turn gives rise to local solutions where an iterative search can be employed but different initializations may end up with different local solutions. Laborious trial and error efforts might hence be incurred in order to reach an acceptable setting.
- The second problem comes from the local plateaus resulting from summing the flat regions of the sigmoid. A lot of search efforts may be spent upon making little progress at these locally flat regions.

Total Error Rate Based Learning

- Fortunately, the optimization problem can be approached from matching the *link-loss functional pair* perspective.
- Mathematically, the TER formulation in (5.65) or (5.67) constitutes a *functional*[1] since it is a composite function, i.e., $L(g(\cdot))$, where L is a loss function and g is a link function.
- By matching the *linear link* ($g(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}$) with a *quadratic loss* ($L(\cdot) = (\cdot)^2$), a convex formulation can be obtained. Particularly, in this linear-quadratic pair, a closed-form solution can be permissible.
- However, we recall that the quadratic function is by no means closed to the loss property of having a heavier penalty for an incorrect output than a correct output where a zero-one function possessed. In other words, a monotonic loss function is necessary for the error counting loss formulation.

[1] In mathematics, a *functional* is often referred to as a *function* of a *function*.

Total Error Rate Based Learning

Nevertheless, from data perspective, we can seek to manipulate the data such that only one quadratic arm is utilized during estimation.

To achieve this goal, a uniform offset can be introduced to the entire bulk of input data so that they are projected onto a single quadratic arm. The dashed curve in Fig. 1 illustrates a shifted quadratic function plotted along with the zero-one step loss function. Translating this idea to our error rate loss function, the following quadratic approximation to (5.67) can be adopted in our minimization formulation:

$$\min_{\boldsymbol{w}} \mathrm{TER}(\boldsymbol{w}) \approx \min_{\boldsymbol{w}} \left\{ \frac{1}{2m^-} \sum_{j=1}^{m^-} (\epsilon_j + \eta)^2 + \frac{1}{2m^+} \sum_{i=1}^{m^+} (\varepsilon_i + \eta)^2 \right\}. \tag{5.71}$$

The normalizing factor of 2 in both the false negative and false positive terms accounts for the differentiation factor resulted from the two square terms.

Total Error Rate Based Learning

Recall that $\epsilon_j := g(\boldsymbol{x}_j^-, \boldsymbol{w}) - \tau$, $j = 1, 2, ..., m^-$ and $\varepsilon_i := \tau - g(\boldsymbol{x}_i^+, \boldsymbol{w}) - \Delta$, $i = 1, 2, ..., m^+$. Given a linear predictor $g(\boldsymbol{x}_i, \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{p}(\boldsymbol{x}_i)$ and considering a weight decay regularization by adding $\frac{b}{2}\|\boldsymbol{w}\|_2^2$, we have

$$\min_{\boldsymbol{w}} \mathrm{TER}(\boldsymbol{w}) \approx \min_{\boldsymbol{w}} \left\{ \frac{b}{2}\|\boldsymbol{w}\|_2^2 + \frac{1}{2m^-}\sum_{j=1}^{m^-}[(\boldsymbol{w}^T \boldsymbol{p}(\boldsymbol{x}_j^-) - \tau) + \eta]^2 + \frac{1}{2m^+}\sum_{i=1}^{m^+}[(\tau - \boldsymbol{w}^T \boldsymbol{p}(\boldsymbol{x}_i^+)) + \eta]^2 \right\}, \tag{5.72}$$

where the value of b controls the amount of regularization.

Total Error Rate Based Learning

The parameter vector $\hat{\boldsymbol{w}}$ which minimizes this approximated TER can be obtained by putting its first partial derivative terms to zeros:

$$\frac{\partial \mathrm{TER}(\boldsymbol{x}^+, \boldsymbol{x}^-, \boldsymbol{w})}{\partial \boldsymbol{w}} = \mathbf{0} \tag{5.73}$$

where it implies that

$$\begin{aligned} b\boldsymbol{w} + \frac{1}{m^-} \sum_{j=1}^{m^-} \boldsymbol{p}(\boldsymbol{x}_j^-) \left[\boldsymbol{p}^T(\boldsymbol{x}_j^-)\boldsymbol{w} - \tau + \eta \right] & \\ + \frac{1}{m^+} \sum_{i=1}^{m^+} -\boldsymbol{p}(\boldsymbol{x}_i^+) \left[\tau - \boldsymbol{p}^T(\boldsymbol{x}_i^+)\boldsymbol{w} + \eta \right] &= \mathbf{0}. \end{aligned} \tag{5.74}$$

Total Error Rate Based Learning

Abbreviating the column basis vectors $\boldsymbol{p}_j = \boldsymbol{p}(\boldsymbol{x}_j^-) \in \mathbb{R}^{D+1}$ and $\boldsymbol{p}_i = \boldsymbol{p}(\boldsymbol{x}_i^+) \in \mathbb{R}^{D+1}$, the solution for $\boldsymbol{w}$ which minimizes (5.72) can be written as

$$\hat{\boldsymbol{w}} = \left[b\mathbf{I} + \frac{1}{m^-} \sum_{j=1}^{m^-} \boldsymbol{p}_j \boldsymbol{p}_j^T + \frac{1}{m^+} \sum_{i=1}^{m^+} \boldsymbol{p}_i \boldsymbol{p}_i^T \right]^{-1} \left[\frac{(\tau - \eta)}{m^-} \sum_{j=1}^{m^-} \boldsymbol{p}_j + \frac{(\tau + \eta)}{m^+} \sum_{i=1}^{m^+} \boldsymbol{p}_i \right] \tag{5.75}$$

where $\mathbf{I}$ is an identity matrix of $(D+1) \times (D+1)$ size (recall that $D+1$ is the total number of transformed feature terms).

Total Error Rate Based Learning

In a more compact matrix notation, (5.75) can be written as:

$$\hat{\boldsymbol{w}} = \left(b\mathbf{I} + \frac{1}{m^-}\mathbf{P}_-^T\mathbf{P}_- + \frac{1}{m^+}\mathbf{P}_+^T\mathbf{P}_+\right)^{-1}\left(\frac{(\tau-\eta)}{m^-}\mathbf{P}_-^T\mathbf{1}_- + \frac{(\tau+\eta)}{m^+}\mathbf{P}_+^T\mathbf{1}_+\right) \tag{5.76}$$

where

$$\mathbf{P}_+ = \begin{bmatrix} \boldsymbol{p}^T(\boldsymbol{x}_1^+) \\ \boldsymbol{p}^T(\boldsymbol{x}_2^+) \\ \vdots \\ \boldsymbol{p}^T(\boldsymbol{x}_{m^+}^+) \end{bmatrix}, \qquad \mathbf{P}_- = \begin{bmatrix} \boldsymbol{p}^T(\boldsymbol{x}_1^-) \\ \boldsymbol{p}^T(\boldsymbol{x}_2^-) \\ \vdots \\ \boldsymbol{p}^T(\boldsymbol{x}_{m^-}^-) \end{bmatrix}, \tag{5.77}$$

and $\mathbf{1}_+ = [1, \ldots, 1]^T \in \mathbb{N}^{m^+}$, $\mathbf{1}_- = [1, \ldots, 1]^T \in \mathbb{N}^{m^-}$.

Total Error Rate Based Learning

Multi-category Learning: Consider the multi-class problem with C output categories. An indicator matrix can be formed for the above TER formulation similar to that in LSE formulation. Assume a common setting for threshold (τ) and bias (η) throughout all C outputs. Subsequently, we have for each kth-class ($k = 1, 2, \ldots, C$), a positive output $\mathbf{y}_k^+ = (\tau + \eta)\mathbf{1}_+ \in \mathbb{R}^{m^+}$ which corresponds to a kth-class indicator, and a negative output $\mathbf{y}_k^- = (\tau - \eta)\mathbf{1}_- \in \mathbb{R}^{m^-}$ which corresponds to a non-kth-class indicator. By packing the solutions with corresponding $\mathbf{P}_{k+}$ and $\mathbf{P}_{k-}$ matrices, equation (5.76) can be extended to multi-category as

$$\hat{\mathbf{W}} = [\hat{\boldsymbol{w}}_1, \cdots, \hat{\boldsymbol{w}}_C], \tag{5.78}$$

where

$$\hat{\boldsymbol{w}}_k = \left(b\mathbf{I} + \frac{1}{m_k^-}\mathbf{P}_{k-}^T\mathbf{P}_{k-} + \frac{1}{m_k^+}\mathbf{P}_{k+}^T\mathbf{P}_{k+} \right)^{-1} \times \left(\frac{1}{m_k^-}\mathbf{P}_{k-}^T\mathbf{y}_k^- + \frac{1}{m_k^+}\mathbf{P}_{k+}^T\mathbf{y}_k^+ \right), \quad k = 1, \ldots, C.$$

Total Error Rate Based Learning

Total Error Rate Based Learning in the Dual Space: By defining two class-specific diagonal weighting matrices

$$\mathbf{M}_{-} = \begin{array}{c} \left[\begin{array}{cccccc} \frac{1}{m^{-}} & & & & & 0 \\ & \ddots & & & & \\ & & \frac{1}{m^{-}} & & & \\ & & & 0 & & \\ & & & & \ddots & \\ 0 & & & & & 0 \end{array}\right] \end{array} \begin{array}{l} \left.\vphantom{\begin{array}{c}1\\1\\1\end{array}}\right\} m^{-} \\ \left.\vphantom{\begin{array}{c}1\\1\\1\end{array}}\right\} m^{+} \end{array} \tag{5.79}$$

$$\mathbf{M}_{+} = \begin{array}{c} \left[\begin{array}{cccccc} 0 & & & & & 0 \\ & \ddots & & & & \\ & & 0 & & & \\ & & & \frac{1}{m^{+}} & & \\ & & & & \ddots & \\ 0 & & & & & \frac{1}{m^{+}} \end{array}\right] \end{array} \begin{array}{l} \left.\vphantom{\begin{array}{c}1\\1\\1\end{array}}\right\} m^{-} \\ \left.\vphantom{\begin{array}{c}1\\1\\1\end{array}}\right\} m^{+} \end{array}, \tag{5.80}$$

Total Error Rate Based Learning

and having the target vector ordered as

$$\mathbf{y} = \begin{bmatrix} \mathbf{y}^- \\ \mathbf{y}^+ \end{bmatrix} = \left[\frac{(\tau - \eta)}{m^-}, \cdots, \frac{(\tau - \eta)}{m^-}, \frac{(\tau + \eta)}{m^+}, \cdots, \frac{(\tau + \eta)}{m^+} \right]^T, \tag{5.81}$$

the Total Error Rate based binary learning can be re-written as:

$$\begin{aligned} \hat{\boldsymbol{w}} &= \left[b\mathbf{I} + \mathbf{P}^T (\mathbf{M}_- + \mathbf{M}_+)\mathbf{P} \right]^{-1} \left[\mathbf{P}^T (\mathbf{M}_- + \mathbf{M}_+)\mathbf{y} \right] \\ &= \left(b\mathbf{I} + \mathbf{P}^T \mathbf{M}\mathbf{P} \right)^{-1} \mathbf{P}^T \mathbf{M}\mathbf{y}, \end{aligned} \tag{5.82}$$

where $\mathbf{M} = \mathbf{M}_- + \mathbf{M}_+$ and the row samples of $\mathbf{P}$ being ordered according to the two classes.

Total Error Rate Based Learning

By putting $\boldsymbol{w} = \mathbf{P}^T\mathbf{M}\boldsymbol{\alpha}$ and using the derivation steps of the dual ridge regression, we have

$$\hat{\boldsymbol{\alpha}} = (b\mathbf{I} + \mathbf{P}\mathbf{P}^T\mathbf{M})^{-1}\mathbf{y}. \tag{5.83}$$

This $\hat{\boldsymbol{\alpha}}$ can be substituted back into $\boldsymbol{w} = \mathbf{P}^T\mathbf{M}\boldsymbol{\alpha}$ to give

$$\hat{\boldsymbol{w}} = \mathbf{P}^T\mathbf{M}(b\mathbf{I} + \mathbf{P}\mathbf{P}^T\mathbf{M})^{-1}\mathbf{y}. \tag{5.84}$$

For multi-category case, the solution for parameter estimation can be packed as:

$$\hat{\mathbf{W}} = [\hat{\boldsymbol{w}}_1, \cdots, \hat{\boldsymbol{w}}_C] \tag{5.85}$$

where

$$\hat{\boldsymbol{w}}_k = \mathbf{P}_k^T\mathbf{M}_k(b\mathbf{I} + \mathbf{P}_k\mathbf{P}_k^T\mathbf{M}_k)^{-1}\mathbf{y}_k, \quad k = 1, 2, \ldots, C. \tag{5.86}$$

Here, it is noted that for each $\hat{\boldsymbol{w}}_k$, $k = 1, \ldots, C$ the sample order of matrices $\mathbf{P}_k$, $\mathbf{M}_k$ needs to be congruence to the sample order of $\mathbf{y}_k$ in terms of their distinct set of indicating target labels for each $k \in \{1, 2, \ldots, C\}$.

Example 5.4

Consider the following training data points for two categories,
$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}: \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}$
which correspond to labels $y : \{1, 0, 1, 0, 0\}$ respectively. Using a 3rd-order polynomial model and the TER method, our goal is to predict the class labels of $\mathbf{x}_{t1} = \begin{bmatrix} 2.5 \\ 4 \end{bmatrix}$ (red 'triangle' in Fig. 2) and $\mathbf{x}_{t2} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ (black 'triangle' in Fig. 2) which have true labels $y_{t1} = 0$, $y_{t2} = 1$ respectively.
Fig. 2 shows the decision boundary at threshold 0 according to the one-hot implementation of the binary target. This threshold is different from that in Example 3 which has an explicit threshold setting at 0.5. The Python codes for TER learning which can operate in either the Primal or the Dual modes are shown below.

Example 5.4

```
from sklearn.preprocessing import PolynomialFeatures
X = np.array([[2,2],[1,2],[1,1],[3,1],[2,3]])
Y = np.array([[1,0],[0,1],[1,0],[0,1],[0,1]])
Xt = np.array([[2.5,4],[1,0]])
Yt = np.array([[0,1],[1,0]])
poly = PolynomialFeatures(3)
P = poly.fit_transform(X)
Pt = poly.fit_transform(Xt)
W_dual = TERtrain(P,Y,0)
Yt_predict, Yt_class_predict = TERtest(Pt, Yt, W_dual)
M1, I1 = np.max(Yt, axis=1), np.argmax(Yt, axis=1)
M2, I2 = np.max(Yt_class_predict, axis=1), np.argmax(Yt_class_predict,
axis=1)
TestAccuracy = (np.mean(I1 == I2))*100
print(Yt_predict)
print(Yt_class_predict)
print(TestAccuracy)
  [[-0.67427395 1.51920227]
  [ 1.17037166 -0.18958447]]
  [[0.  1.]
  [1.  0.]]
  100.0
```

Example 5.4 Cont'd

```
def TERtrain(P,Y,b=None,Sp=None,Sm=None):
 # Training by TER method Primal/Dual
 if b is None and Sp is None and Sm is None:
  # 'b' and 'Sp, Sm' scaling not given
  Sp=1; Sm=1; b = 1e-4 # Use default settings
 elif Sp is None and Sm is None:
  # only 'Sp, Sm' scaling not given
  Sp=1; Sm=1
 m,K = Y.shape
 W = np.zeros((P.shape[1], K))
 for jj in range(K):
  max_value = np.max(Y[:, jj])
  min_value = np.min(Y[:, jj])
  Mp, D = P[Y[:,jj]==max_value,:].shape
  Mm, D = P[Y[:,jj]==min_value,:].shape
  Mvector = np.ones(m)
  Mvector[Y[:,jj]==min_value] = 1/(Sm*Mm)
  Mvector[Y[:,jj]==max_value] = 1/(Sp*Mp)
  M = np.diag(Mvector)
  if m>=D: # Primal
    II = np.eye(D,D)
    W[:,jj] = inv(P.T @ M @ P + b*II) @ P.T @ M @ Y[:,jj]
  else:  # Dual
    II = np.eye(m,m)
    W[:,jj] = P.T @ inv(M @ P @ P.T + b*II) @ M @ Y[:,jj]
  return W
```

Example 5.4 Cont'd

```
def TERtest(Pt, Yt, W):
 #- prediction -%
 Yt_out_pred = Pt @ W
 nrow, ncol = Yt.shape
 if ncol == 1:
  #- Binary -%
  Yt_class_pred = np.zeros((nrow, 1))
  Yt_class_pred[Yt_out_pred>=(np.max(Yt)+np.min(Yt))/2] = np.max(Yt)
  Yt_class_pred[Yt_out_pred<(np.max(Yt)+np.min(Yt))/2] = np.min(Yt)
  TestAccuracy = 100 * len(np.where(Yt == Yt_class_pred)[0])/nrow
 else:
  #- one-versus-all -%
  Yt_class_pred = np.zeros((nrow,ncol))
  for jj in range(nrow):
    jindx = np.argmax(Yt_out_pred[jj,:]),np.argmax(Yt_out_pred[jj,:])
    Yt_class_pred[jj,jindx] = 1
  M1, I1 = np.max(Yt, axis=1), np.argmax(Yt, axis=1)
  M2, I2 = np.max(Yt_class_pred, axis=1), np.argmax(Yt_class_pred, axis=1)
  TestAccuracy = (np.mean(I1 == I2))*100
  return Yt_out_pred, Yt_class_pred, TestAccuracy
```

Example 5.4 Cont'd

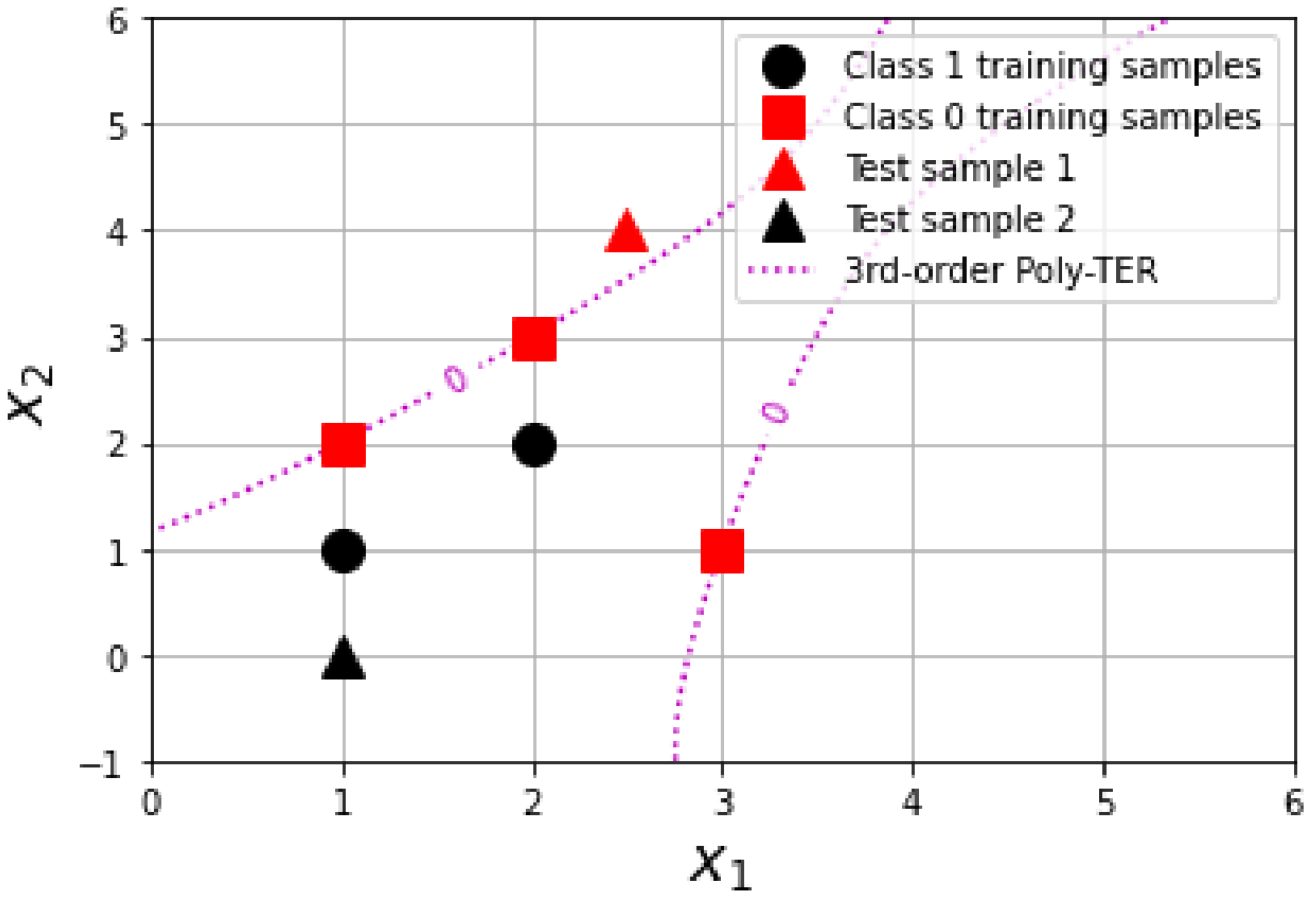


Figure 2: Learning decision contour at level [0].

AUC Based Learning

Binary Learning:

- Apart from the error rate based cost function measured at single operating threshold, the receiver operating characteristic (ROC) curve provides an overview of error costs over all operating thresholds which can also be utilized for learning.
- However, the ROC gives a range of operating points instead of a scalar value, it is not convenient for optimization.
- The Area Under the ROC Curve (AUC) is thus a natural alternative as the learning cost function for classifier learning concerning all operating points.

AUC Based Learning

For binary data samples indexed by $i = 1, 2, \ldots, m^+$ and $j = 1, 2, \ldots, m^-$ in respective categories, let us denote the difference between the predictor values of a positive sample $\boldsymbol{x}_i^+$ and that of a negative sample $\boldsymbol{x}_j^-$ by $\xi_{ij} = g(\boldsymbol{x}_i^+) - g(, \boldsymbol{x}_j^-)$. Here, $\xi_{ij} > 0$ indicates that the positive sample $\boldsymbol{x}_i^+$ and the negative sample $\boldsymbol{x}_j^-$ are correctly ranked by the predictor g. Consider the Heaviside step function given by

$$u(\xi) = \begin{cases} 1, & \text{if } \xi > 0, \\ 0.5 & \text{if } \xi = 0, \\ 0, & \text{if } \xi < 0. \end{cases} \tag{5.87}$$

Then, the AUC for $m = m^+ + m^-$ training examples can be expressed as:

$$\begin{aligned} \mathrm{AUC}(\boldsymbol{w}, \boldsymbol{x}) &= \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(\xi_{ij}) \\ &= \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(g(\boldsymbol{x}_i^+) - g(, \boldsymbol{x}_j^-)), \end{aligned} \tag{5.88}$$

which is the fraction of positive-negative pairs that are ranked correctly by predictor $g(\cdot)$. The value in (5.88) is also known as the Wilcoxon-Mann-Whitney statistic.

AUC Based Learning

The AUC can be maximized with respect to $\boldsymbol{w}$ of the parametric predictor $g(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{p}(\boldsymbol{x})$ for effective classification considering the entire operating threshold range. Alternatively, the following equivalent minimization of the *Area Above the ROC Curve* (AAC) can be used:

$$\min_{\boldsymbol{w}} \mathrm{AAC}(\boldsymbol{w}, \boldsymbol{x}) = \min_{\boldsymbol{w}} \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} u(-\xi_{ij}) \quad . \tag{5.89}$$

Similar to the TER approximation, when $g(\cdot)$ is in linear parametric form, the minimization problem given by (5.89) can be solved by a quadratic approximation to the non-differentiable step function $u(\cdot)$ with a weight decay such as

$$\begin{aligned} &\min_{\boldsymbol{w}} \mathrm{AAC}(\boldsymbol{w}, \boldsymbol{x}) \\ &= \min_{\boldsymbol{w}} \left\{ \frac{b}{2} \|\boldsymbol{w}\|_2^2 + \frac{1}{2\, m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left[\left(\boldsymbol{p}(\boldsymbol{x}_j^-) - \boldsymbol{p}(\boldsymbol{x}_i^+) \right)^T \boldsymbol{w} + \eta \right]^2 \right\} . \end{aligned} \tag{5.90}$$

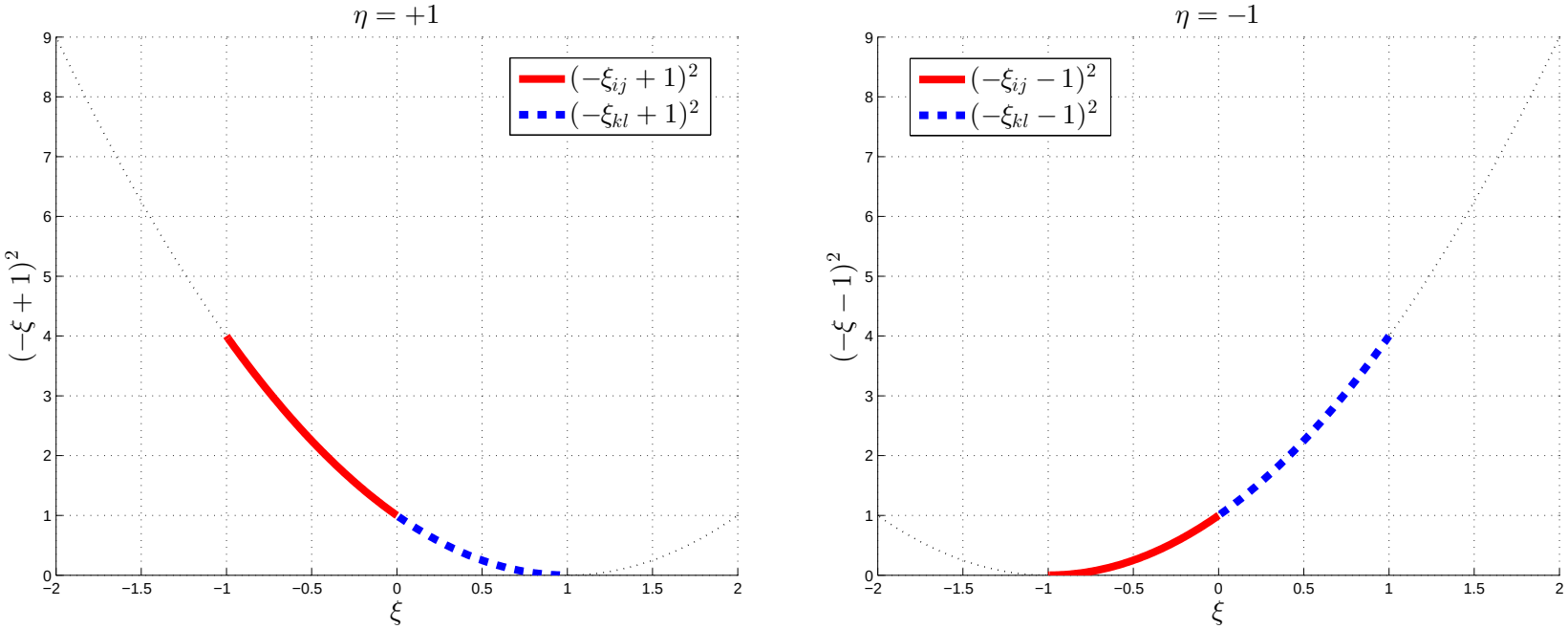
AUC Based Learning

- When the predictor is normalized so that $g \in [0, 1]$, then the ranking difference value is $\xi_{ij} \in [-1, +1]$.
- An effective approximation to (5.89) requires a monotonic function for the ranking process, i.e., a cost function should give a 'high' output whenever $-\xi_{ij} = g(\boldsymbol{w}, \boldsymbol{x}_i^-) - g(\boldsymbol{w}, \boldsymbol{x}_j^+) > 0$ (wrongly ranked) and a 'low' output whenever $-\xi_{ij} = g(\boldsymbol{w}, \boldsymbol{x}_i^-) - g(\boldsymbol{w}, \boldsymbol{x}_j^+) < 0$ (correctly ranked).
- However, taking a quadratic on $-\xi_{ij}$ alone will always result in a 'high' output regardless of its sign.
- The situation can be managed, however, by including an offset η to the quadratic function, such that $(-\xi_{ij} + \eta)^2 > (-\xi_{kl} + \eta)^2$ whenever $\xi_{ij} < 0$ and $\xi_{kl} > 0$, for all ξ_{ij} and ξ_{kl} in $[-1, +1]$.

AUC Based Learning

- Fig. 3(a) shows an illustration of the quadratic output values of those wrongly ranked data ($\xi_{ij} < 0$, which is indicated by solid-line) being higher than those generated from correctly ranked data ($\xi_{kl} > 0$, which is indicated by dashed-line), i.e., $(-\xi_{ij} + \eta)^2 > (-\xi_{kl} + \eta)^2$ whenever $\xi_{ij} < 0$ and $\xi_{kl} > 0$ when $\eta = +1$.
- Conversely, when $\eta = -1$ in Fig. 3(b), those wrongly ranked data ($\xi_{ij} < 0$, indicated by solid-line) give lower quadratic output values $(-\xi_{ij} + \eta)^2$ than those given by $(-\xi_{kl} + \eta)^2$, which are generated by the correctly ranked data ($\xi_{kl} > 0$, indicated by dashed-line). This means that only one arm of the quadratic function has been utilized for monotonic ranking approximation.

AUC Based Learning



(a) $\eta = +1$: solid-line > dashed-line (b) $\eta = -1$: dashed-line < solid-line

Figure 3: (a) $\eta = +1$: $(-\xi_{ij}+\eta)^2 > (-\xi_{kl}+\eta)^2$ whenever $\xi_{ij} < 0$ and $\xi_{kl} > 0$; (b) $\eta = -1$: $(-\xi_{ij}+\eta)^2 < (-\xi_{kl}+\eta)^2$ whenever $\xi_{ij} < 0$ and $\xi_{kl} > 0$.

AUC Based Learning

Under this setting, the minimization in (5.90) can be solved by setting its gradient with respect to $\boldsymbol{w}$ equals to zero, giving

$$\hat{\boldsymbol{w}} = \left[b\mathbf{I} + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\boldsymbol{p}_j^- - \boldsymbol{p}_i^+ \right) \left(\boldsymbol{p}_j^- - \boldsymbol{p}_i^+ \right)^T \right]^{-1} \times \left[\frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\boldsymbol{p}_j^- - \boldsymbol{p}_i^+ \right) \right], \quad (5.91)$$

with the projection column vectors being abbreviated by $\boldsymbol{p}_j^- := \boldsymbol{p}(\boldsymbol{x}_j^-)$ and $\boldsymbol{p}_i^+ := \boldsymbol{p}(\boldsymbol{x}_i^+)$.

AUC Based Learning

- The solution $\hat{\boldsymbol{w}}$ in (5.91) for optimal AUC classification decision can be obtained in a single evaluation step which is also least-squares optimal but in the AUC sense.
- This solution can be used to compute the outputs for an unseen test data set $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ based on the stacked projection matrix $\mathbf{P}_n$ as that in the least squares regression, i.e., $\hat{\boldsymbol{g}}(\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \mathbf{P}_n \hat{\boldsymbol{w}}$.

AUC Based Learning

The above solution for AUC or AAC is optimal with respect to the full range of operating threshold decision points. Hence, there is no explicit threshold being defined in (5.91). However, in practice, a specific choice of threshold is often needed for decision making. To do this, an optimal threshold with respect to the total error rate can be obtained by minimizing the approximated TER (5.71) with respect to τ and utilizing the above AAC solution (5.91), giving

$$\tau = \frac{1}{2m^-} \sum_{j=1}^{m^-} \hat{\boldsymbol{w}}^T \boldsymbol{p}_j^- + \frac{1}{2m^+} \sum_{i=1}^{m^+} \hat{\boldsymbol{w}}^T \boldsymbol{p}_i^+ . \tag{5.92}$$

In summary, the AAC minimizer (5.91) provides an optimal solution for all operating classification thresholds. This is useful when the threshold needs to be adjusted without compromising the optimality in terms of the overall operating performance. To cater for specific operating requirements, an optimal threshold with respect to minimizing the TER can nevertheless be chosen based on (5.92).

Example 5.5

Consider the following training data points for two categories,
$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}: \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}$
which correspond to labels $y : \{1, 0, 1, 0, 0\}$ respectively. Our goal here is use AUC learning to predict the class labels of $\mathbf{x}_{t1} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$ (red 'triangle' in Fig. 2) and $\mathbf{x}_{t2} = \begin{bmatrix} 3 \\ 0.3 \end{bmatrix}$ (black 'triangle' in Fig. 2) which have true labels $y_{t1} = 0$, $y_{t2} = 1$ respectively.

Example 5.5 Cont'd

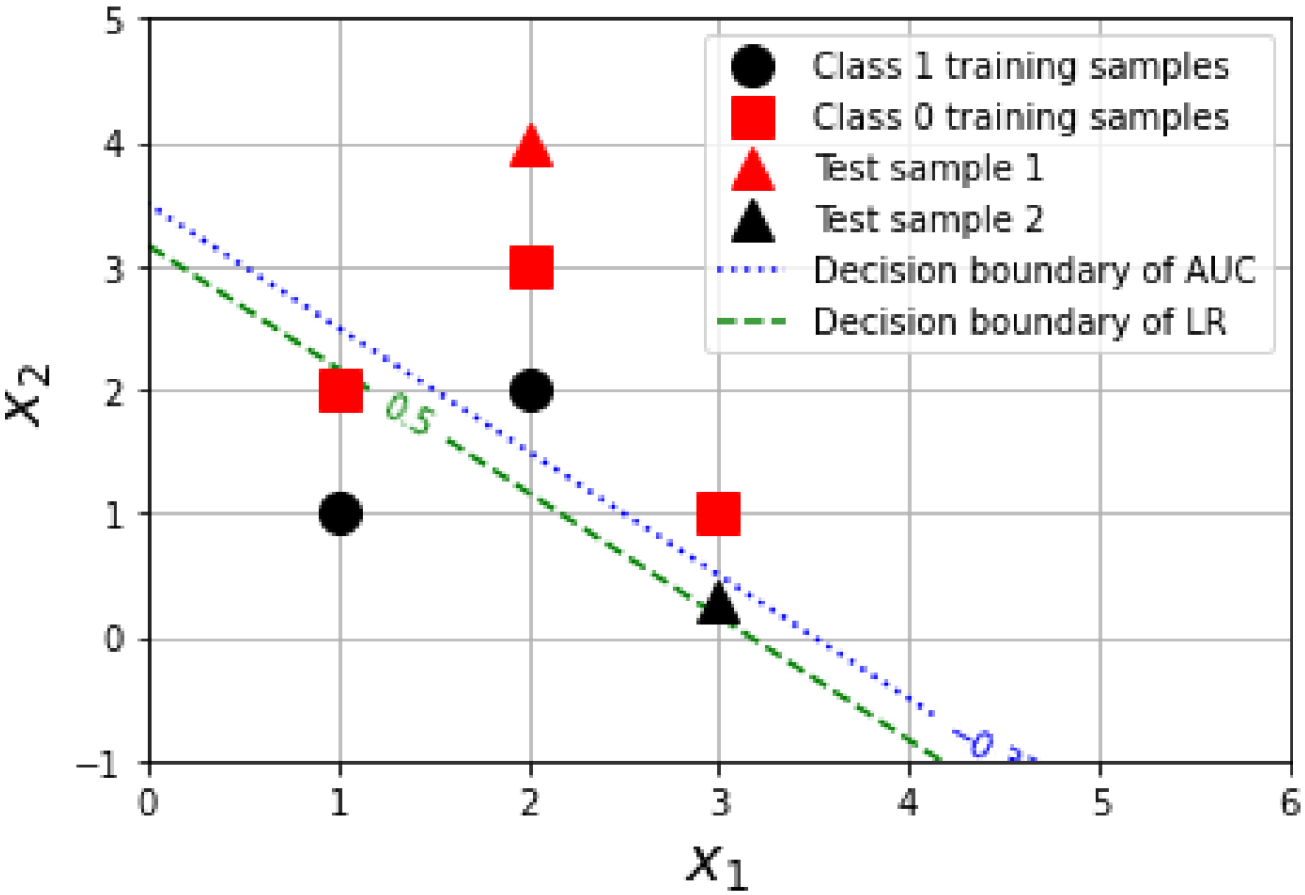


Figure 4: Learning decision contours.

Data Transformation

- Based on the structural resemblance among the LSE, TER and AUC solutions, it is observed that these classifiers can be linked together by simple data manipulation.
- Here, we introduce an additive form of linearly transformed data at feature level for the AUC formulation.
- By mere variation of coefficient settings of the additive terms, we show that several linear classifiers can be interrelated.
- The knowledge of this relationship leads to a generalized classifier which allows variation of its coefficient settings beyond the above known settings.

Data Transformation

Consider each sample of input feature vector given by $\boldsymbol{x} \in \mathbb{R}^d$ with a corresponding target label given by $y \in \{0, 1\}$, a transformation on the data $\boldsymbol{x}$ can be performed as follows:

$$\tilde{\boldsymbol{x}} = \boldsymbol{T}\boldsymbol{x} + \boldsymbol{a}, \tag{5.93}$$

where $\boldsymbol{a} = [a_1, a_2, ..., a_d]^T \in \mathbb{R}^d$ is a column vector for translation and $\boldsymbol{T}$ is a $d \times d$ matrix.

Next consider m number of training samples given by $(\boldsymbol{x}_i, y_i)$, where $i = 1, 2, \ldots, m$. Suppose there are K distinct scaling matrices $\boldsymbol{T}_k$ and translating vectors $\boldsymbol{a}_k$, $k = 1, 2, ..., K$. This gives rise to $m \times K$ training data samples:

$$\tilde{\boldsymbol{x}}_{ik} = \boldsymbol{T}_k \boldsymbol{x}_i + \boldsymbol{a}_k, i = 1, ..., m, \ k = 1, ..., K. \tag{5.94}$$

In a similar fashion, we could also work at the transformed projection vectors of the generalized linear models:

$$\tilde{\boldsymbol{p}}(\boldsymbol{x}) = \mathbf{T}\boldsymbol{p}(\boldsymbol{x}) + \mathbf{a}, \tag{5.95}$$

where $\boldsymbol{p}(\boldsymbol{x}) \in \mathbb{R}^{D+1}$ projects $\boldsymbol{x}$ into a nonlinear hyperplane, $\mathbf{T}$ is $(D + 1) \times (D + 1)$ matrix and $\mathbf{a} \in \mathbb{R}^{D+1}$.

Data Transformation

Now, consider a linear parametric model applied respectively to the transformed data in (5.94) and the transformed projection vector (5.95), we have

$$g(\boldsymbol{w}, \tilde{\boldsymbol{x}}) = \sum_{j=1}^{d} w_j \tilde{\boldsymbol{x}}_j = \tilde{\boldsymbol{x}}^T \boldsymbol{w} \tag{5.96}$$

and

$$\tilde{g}(\boldsymbol{w}, \boldsymbol{x}) = \sum_{j=1}^{D+1} w_j \tilde{p}_j(\boldsymbol{x}) = \tilde{\boldsymbol{p}}(\boldsymbol{x})^T \boldsymbol{w} \tag{5.97}$$

respectively.

Data Transformation

In the following discussion, we shall use (5.97) for our development while noticing that a similar deployment can be carried out by replacing $\tilde{\boldsymbol{p}}(\boldsymbol{x})$ by $\tilde{\boldsymbol{x}}$. To simplify the notations, we shall use

$$\tilde{\boldsymbol{p}}_{kj}^{-} = \tilde{\boldsymbol{p}}_k(\boldsymbol{x}_j^{-}) = \mathbf{T}_k \boldsymbol{p}(\boldsymbol{x}_j^{-}) + \mathbf{a}_k,$$
$$\tilde{\boldsymbol{p}}_{ki}^{+} = \tilde{\boldsymbol{p}}_k(\boldsymbol{x}_i^{+}) = \mathbf{T}_k \boldsymbol{p}(\boldsymbol{x}_i^{+}) + \mathbf{a}_k,$$

for $k = 1, 2, \ldots, K$, $j = 1, 2, \ldots, m^{-}$, and $i = 1, 2, \ldots, m^{+}$.

Data Transformation

The new AUC-based (AAC, to be specific) learning formulation based on Equation (5.97) is thus

$$\min_{\boldsymbol{w}} \mathrm{A\tilde{A}C}(\boldsymbol{w}, \boldsymbol{x}^{+}, \boldsymbol{x}^{-}) = \min_{\boldsymbol{w}} \left\{ \frac{b}{2} \|\boldsymbol{w}\|_2^2 + \frac{1}{2\, m^{+} m^{-}} \sum_{i=1}^{m^{+}} \sum_{j=1}^{m^{-}} \sum_{k=1}^{K} \left[\left(\tilde{\boldsymbol{p}}_{kj}^{-} - \tilde{\boldsymbol{p}}_{ki}^{+} \right)^{T} \boldsymbol{w} + \eta \right]^2 \right\} . \quad (5.98)$$

Data Transformation

Then, according to the AUC solution (26), the solution $\tilde{\boldsymbol{w}}_{\text{TAUC}}$ which minimizes the Transformed AUC (TAUC) in (5.98) is given by

$$\tilde{\boldsymbol{w}}_{\text{TAUC}} = \left[b\mathbf{I} + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \sum_{k=1}^{K} \left(\tilde{\boldsymbol{p}}_{kj}^- - \tilde{\boldsymbol{p}}_{ki}^+ \right) \left(\tilde{\boldsymbol{p}}_{kj}^- - \tilde{\boldsymbol{p}}_{ki}^+ \right)^T \right]^{-1} \times \left[\frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \sum_{k=1}^{K} \left(\tilde{\boldsymbol{p}}_{kj}^- - \tilde{\boldsymbol{p}}_{ki}^+ \right) \right]. \tag{5.99}$$

Based on this generalized solution, we shall observe the relationships among several existing binary classifiers and predictors in the following subsections.

Between TAUC and AUC Solutions

Consider a transformation with $\mathbf{T}_k = ((1-y)\beta_k + y\gamma_k)\mathbf{I}_{D+1}$ (where $\mathbf{I}_{D+1} \in \mathbb{R}^{(D+1)\times(D+1)}$) which is dependent on the label $y \in \{0, 1\}$, where $k = 1, 2, \ldots, K$. In other words the transformation uses the scaling transformation β_k when $y = 0$ and it uses γ_k when $y = 1$. Writing this transformation in terms of negative and positive labels, we have

$$\begin{aligned} \tilde{\boldsymbol{p}}_{kj}^{-} &= \tilde{\boldsymbol{p}}_k(\boldsymbol{x}_j^{-}) = \beta_k \boldsymbol{p}(\boldsymbol{x}_j^{-}) + \mathbf{a}_k, \\ \tilde{\boldsymbol{p}}_{ki}^{+} &= \tilde{\boldsymbol{p}}_k(\boldsymbol{x}_i^{+}) = \gamma_k \boldsymbol{p}(\boldsymbol{x}_i^{+}) + \mathbf{a}_k. \end{aligned} \tag{5.100}$$

This means that each β_k corresponds to a global scaling of the negative samples while each γ_k corresponds to a global scaling of the positive samples.

Between TAUC and AUC Solutions

When $K = 2$ (i.e., with two scaling transformations) and $\mathbf{a}_k = \mathbf{0}$ (which means no translation), the AUC solution that incorporates only scaling transformation can be written as:

$$\tilde{\boldsymbol{w}} = \left[b\mathbf{I} + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_1 \boldsymbol{p}_j^- - \gamma_1 \boldsymbol{p}_i^+\right) \left(\beta_1 \boldsymbol{p}_j^- - \gamma_1 \boldsymbol{p}_i^+\right)^T \right.$$
$$\left. + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_2 \boldsymbol{p}_j^- - \gamma_2 \boldsymbol{p}_i^+\right) \left(\beta_2 \boldsymbol{p}_j^- - \gamma_2 \boldsymbol{p}_i^+\right)^T \right]^{-1}$$
$$\times \left[\frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_1 \boldsymbol{p}_j^- - \gamma_1 \boldsymbol{p}_i^+\right) + \frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_2 \boldsymbol{p}_j^- - \gamma_2 \boldsymbol{p}_i^+\right) \right]. \quad (5.101)$$

Here we see that the AUC solution (5.91) becomes a special case of (5.101) when $\beta_1 = \beta_2 = \gamma_1 = \gamma_2 = 1$. In other words, the TAUC solution generalizes the AUC solution by considering scaled data in the learning.

Between TAUC and TER Solutions

Now, consider another specific case for $K = 2$ with $\gamma_1 = \beta_2 = 0$ and $\beta_1 = \gamma_2 = 1$, we have

$$\tilde{\boldsymbol{w}} = \left[b\mathbf{I} + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_1 \boldsymbol{p}_j^-\right) \left(\beta_1 \boldsymbol{p}_j^-\right)^T + \frac{1}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(-\gamma_2 \boldsymbol{p}_i^+\right) \left(-\gamma_2 \boldsymbol{p}_i^+\right)^T \right]^{-1}$$
$$\times \left[\frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(\beta_1 \boldsymbol{p}_j^-\right) + \frac{-\eta}{m^+ m^-} \sum_{i=1}^{m^+} \sum_{j=1}^{m^-} \left(-\gamma_2 \boldsymbol{p}_i^+\right) \right]$$
$$= \left[b\mathbf{I} + \frac{1}{m^-} \sum_{j=1}^{m^-} \boldsymbol{p}_j^- \boldsymbol{p}_j^{-T} + \frac{1}{m^+} \sum_{i=1}^{m^+} \boldsymbol{p}_i^+ \boldsymbol{p}_i^{+T} \right]^{-1} \left[\frac{0-\eta}{m^-} \sum_{j=1}^{m^-} \boldsymbol{p}_j^- + \frac{0+\eta}{m^+} \sum_{i=1}^{m^+} \boldsymbol{p}_i^+ \right] \quad (5.102)$$

Except for a default zero threshold ($\tau = 0$), the above solution (5.102) is similar to that of TER.

Between TAUC and FLD Solutions

Consider, the scaled version of (5.101) plus a global translation for each class, i.e.,

$$\tilde{\boldsymbol{p}}_k(\boldsymbol{x}) = ((1-y)\beta_k + y\gamma_k)\mathbf{I}_{D+1}\boldsymbol{p}(\boldsymbol{x}) + (1-y)\boldsymbol{u}_k + y\boldsymbol{v}_k, \text{where } k = 1, 2,$$

for each sample pair $(\boldsymbol{x}, y)$), we get:

$$\begin{aligned}\tilde{\boldsymbol{w}} = \Bigg[& b\mathbf{I} + \frac{1}{m^+m^-}\sum_{i=1}^{m^+}\sum_{j=1}^{m^-}\left((\beta_1\boldsymbol{p}_j^- - \boldsymbol{u}_1) - (\gamma_1\boldsymbol{p}_i^+ - \boldsymbol{v}_1)\right)\left((\beta_1\boldsymbol{p}_j^- - \boldsymbol{u}_1) - (\gamma_1\boldsymbol{p}_i^+ - \boldsymbol{v}_1)\right)^T \\ & + \frac{1}{m^+m^-}\sum_{i=1}^{m^+}\sum_{j=1}^{m^-}\left((\beta_2\boldsymbol{p}_j^- - \boldsymbol{u}_2) - (\gamma_2\boldsymbol{p}_i^+ - \boldsymbol{v}_2)\right)\left((\beta_2\boldsymbol{p}_j^- - \boldsymbol{u}_2) - (\gamma_2\boldsymbol{p}_i^+ - \boldsymbol{v}_2)\right)^T\Bigg]^{-1} \\ \times \Bigg[& \frac{-\eta}{m^+m^-}\sum_{i=1}^{m^+}\sum_{j=1}^{m^-}\left((\beta_1\boldsymbol{p}_j^- - \boldsymbol{u}_1) - (\gamma_1\boldsymbol{p}_i^+ - \boldsymbol{v}_1)\right) + \frac{-\eta}{m^+m^-}\sum_{i=1}^{m^+}\sum_{j=1}^{m^-}\left((\beta_2\boldsymbol{p}_j^- - \boldsymbol{u}_2) - (\gamma_2\boldsymbol{p}_i^+ - \boldsymbol{v}_2)\right)\Bigg].\end{aligned} \tag{5.103}$$

Between TAUC and FLD Solutions

When $b = \gamma_1 = \beta_2 = 0$, $\beta_1 = \gamma_2 = 1$ and $\eta = 1/2$, by setting $\boldsymbol{u}_2 = \boldsymbol{v}_1 = \mathbf{0}$, $\boldsymbol{u}_1 = \frac{1}{m^-}\sum_{j=1}^{m^-} \boldsymbol{p}_i^- = \boldsymbol{\mu}_-$ (mean of vectors $\boldsymbol{p}_i^-$) while $\boldsymbol{v}_2 = \frac{1}{m^+}\sum_{j=1}^{m^+} \boldsymbol{p}_j^+ = \boldsymbol{\mu}_+$ (mean of vectors $\boldsymbol{p}_j^+$), we arrive at

$$\tilde{\boldsymbol{w}} = \left[\frac{1}{m^-}\sum_{j=1}^{m^-}(\boldsymbol{p}_j^- - \boldsymbol{\mu}_-)(\boldsymbol{p}_j^- - \boldsymbol{\mu}_-)^T + \frac{1}{m^+}\sum_{i=1}^{m^+}(\boldsymbol{p}_i^+ - \boldsymbol{\mu}_+)(\boldsymbol{p}_i^+ - \boldsymbol{\mu}_+)^T\right]^{-1} \times (\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)\,. \tag{5.104}$$

In particular, when $m^- = m^+ = 1$, the above solution is analogous to Fisher Linear Discriminant analysis (FLD, see [Duda et al., 2001], p.120), but with embedding of nonlinear features within basis expansion vector $\boldsymbol{p}$ (i.e., $\boldsymbol{p}(\boldsymbol{x})$ instead of $\boldsymbol{x}$). Hence the solution to the transformed AUC space can be generalized to contain the FLD solution. Moreover, the FLD solution can be considered a centered version of the TER solution.

Between TER and LSE Solutions

Utilizing the weight matrices $\mathbf{M}_-, \mathbf{M}_+$ and without considering the weight decay regularization, the TER solution (5.102) can be written using matrix notation, i.e.

$$\begin{aligned} \tilde{\boldsymbol{w}} &= \left[\mathbf{P}^T(\mathbf{M}_- + \mathbf{M}_+)\mathbf{P}\right]^{-1}\left[\mathbf{P}^T(\mathbf{M}_- + \mathbf{M}_+)\mathbf{y}\right] \\ &= \left(\mathbf{P}^T\mathbf{M}\mathbf{P}\right)^{-1}\mathbf{P}^T\mathbf{M}\mathbf{y}, \end{aligned} \tag{5.105}$$

where $\mathbf{M} = \mathbf{M}_- + \mathbf{M}_+$ and elements of $\mathbf{P}$ being ordered according to the two classes. The column vector $\mathbf{y}$ is the target vector given by

$$\left[\frac{(0-\eta)}{m^-}, \cdots, \frac{(0-\eta)}{m^-}, \frac{(0+\eta)}{m^+}, \cdots, \frac{(0+\eta)}{m^+}\right]^T. \tag{5.106}$$

Hence, the TER solution can be further related to a Weighted Least Squares (Weighted-LS) solution. Together with the well-known relationship between LS and Weighted-LS, this observation shows that the AUC in the scaling space (Equation (5.101)) provides a generalized framework for estimation based on TER, Weighted-LS and LS.

Relationship with Bayesian Inference

In statistics, the regression model is often analyzed within the context of Bayesian inference with particular assumption of likelihood and prior probability distributions. Consider a data set of input-target pairs $(\boldsymbol{x}_i, y_i)$, $i = 1, 2, \ldots, m$ and assume that the targets are samples from the model with additive noise e_i:

$$y_i = g(\boldsymbol{x}_i, \boldsymbol{w}) + e_i = \boldsymbol{x}_i^T \boldsymbol{w} + e_i, \tag{5.107}$$

where $e_i = y_i - g(\boldsymbol{x}_i, \boldsymbol{w})$ being independent samples drawn from a certain noise process which is assumed to be Gaussian with zero mean and variance σ^2 denoted as $\mathcal{N}(0, \sigma^2)$. Assume independence of samples y_i, the corresponding likelihood of the complete data set can be written as:

$$\begin{aligned} p(\mathbf{y}|\mathbf{X}, \boldsymbol{w}) &= \prod_{i=1}^{m} p(y_i|\boldsymbol{x}_i, \boldsymbol{w}) \\ &= (2\pi\sigma^2)^{-m/2} \exp\left\{-\frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\boldsymbol{w})^T(\mathbf{y} - \mathbf{X}\boldsymbol{w})\right\}, \end{aligned} \tag{5.108}$$

where $\mathbf{y}$, $\mathbf{X}$ and $\boldsymbol{w}$ are defined similarly to those stacked symbols in LSE.

Relationship with Bayesian Inference

This likelihood function for the data is next combined with a prior belief (frequently chosen again, to be Gaussian such as $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_p)$) about the parameters according to Bayes' theorem to yield the posterior belief about the parameter $\boldsymbol{w}$:

$$p(\boldsymbol{w}|\mathbf{X}, \mathbf{y}) \sim \mathcal{N}(\bar{\boldsymbol{w}}, \boldsymbol{\Sigma}), \tag{5.109}$$

where $\bar{\boldsymbol{w}} = \frac{1}{\sigma^2}(\frac{1}{\sigma^2}\mathbf{P}^T\mathbf{P} + \boldsymbol{\Sigma}_p^{-1})^{-1}\mathbf{P}^T\mathbf{y}$ and $\boldsymbol{\Sigma} = (\frac{1}{\sigma^2}\mathbf{P}^T\mathbf{P} + \boldsymbol{\Sigma}_p^{-1})^{-1}$.

Relationship with Bayesian Inference

Recall the TER formulation with quadratic approximation, which can be written in residual form as

$$\begin{aligned} \text{TER} &= \frac{1}{m^-}\sum_{j=1}^{m^-} L(\epsilon_j) + \frac{1}{m^+}\sum_{i=1}^{m^+} L(\varepsilon_i) \\ &\approx \frac{1}{m^-}\sum_{j=1}^{m^-} (\epsilon_j + \eta)^2 + \frac{1}{m^+}\sum_{i=1}^{m^+} (\varepsilon_i + \eta)^2, \end{aligned} \tag{5.110}$$

where $\epsilon_j = g(\boldsymbol{x}_j^-) - \tau$ and $\varepsilon_i = \tau - g(\boldsymbol{x}_i^+)$ can both be assumed Gaussian with independency among samples within and between the two classes. Consider a linear projection model for g and let $y_j^- = \tau - \eta$ and $y_i^+ = \tau + \eta$, the above equation can be rewritten as:

$$\text{TER} \approx \frac{1}{m^-}\sum_{j=1}^{m^-} \left((\boldsymbol{x}_i^-)^T \boldsymbol{w} - y_j^-\right)^2 + \frac{1}{m^+}\sum_{i=1}^{m^+} \left(y_i^+ - (\boldsymbol{x}_i^+)^T \boldsymbol{w}\right)^2. \tag{5.111}$$

Relationship with Bayesian Inference

Unlike common regression, here y_j^-, $j = 1, ..., m^-$ and y_i^+, $i = 1, ..., m^+$ can be thought of as two constant target values corresponding to the two classes. Optimization of TER by quadratic approximation in (5.111) can thus be treated as a *dual-Gaussian* regression:

$$\begin{aligned} p(\mathbf{y}|\mathbf{X}, \boldsymbol{w}) &= \left(\prod_{j=1}^{m^-} p(y_j^-|\boldsymbol{x}_j^-, \boldsymbol{w})\right)\left(\prod_{i=1}^{m^+} p(y_i^+|\boldsymbol{x}_i^+, \boldsymbol{w})\right) \\ &= (2\pi\sigma_-^2)^{-m^-/2} \exp\left\{-\frac{1}{2\sigma_-^2}(\mathbf{y}^- - \mathbf{X}_-\boldsymbol{w})^T(\mathbf{y}^- - \mathbf{X}_-\boldsymbol{w})\right\} \\ &\quad \times (2\pi\sigma_+^2)^{-m^+/2} \exp\left\{-\frac{1}{2\sigma_+^2}(\mathbf{y}^+ - \mathbf{X}_+\boldsymbol{w})^T(\mathbf{y}^+ - \mathbf{X}_+\boldsymbol{w})\right\}. \end{aligned} \tag{5.112}$$

Relationship with Bayesian Inference

By Bayes' rule, the posterior probability of parameter estimate can be written as

$$
\begin{aligned}
p(\boldsymbol{w}|\mathbf{y},\mathbf{X}) &= \frac{p(\mathbf{y}|\mathbf{X},\boldsymbol{w})p(\boldsymbol{w})}{p(\mathbf{y}|\mathbf{X})} \\
&\propto \exp\left\{-\frac{1}{2\sigma_-^2}(\mathbf{y}^- - \mathbf{X}_-\boldsymbol{w})^T(\mathbf{y}^- - \mathbf{X}_-\boldsymbol{w})\right\} \\
&\quad \times \exp\left\{-\frac{1}{2\sigma_+^2}(\mathbf{y}^+ - \mathbf{X}_+\boldsymbol{w})^T(\mathbf{y}^+ - \mathbf{X}_+\boldsymbol{w})\right\}\exp\left(-\frac{1}{2}\boldsymbol{w}^T\Sigma_p^{-1}\boldsymbol{w}\right).
\end{aligned}
\tag{5.113}
$$

Relationship with Bayesian Inference

By multiplying out the product terms and removing the zero mean terms, the previous equation reduces to

$$p(\boldsymbol{w}\,|\mathbf{y},\mathbf{X}) \propto \exp\left(-\frac{1}{2}(\boldsymbol{w}-\bar{\boldsymbol{w}})^T\left(\frac{1}{\sigma_-^2}\mathbf{X}_-^T\mathbf{X}_- + \frac{1}{\sigma_+^2}\mathbf{X}_+^T\mathbf{X}_+ + \Sigma_p^{-1}\right)(\boldsymbol{w}-\bar{\boldsymbol{w}})\right), \tag{5.114}$$

where

$$\bar{\boldsymbol{w}} = \left(\frac{1}{\sigma_-^2}\mathbf{X}_-^T\mathbf{X}_- + \frac{1}{\sigma_+^2}\mathbf{X}_+^T\mathbf{X}_+ + \Sigma_p^{-1}\right)^{-1}\left(\frac{1}{\sigma_-^2}\mathbf{X}_-^T\mathbf{y}^- + \frac{1}{\sigma_+^2}\mathbf{X}_+^T\mathbf{y}^+\right), \tag{5.115}$$

and the covariance is

$$\boldsymbol{\Sigma} = (\frac{1}{\sigma_-^2}\mathbf{X}_-^T\mathbf{X}_- + \frac{1}{\sigma_+^2}\mathbf{X}_+^T\mathbf{X}_+ + \Sigma_p^{-1})^{-1}. \tag{5.116}$$

Relationship with Bayesian Inference

The mean value given by $\bar{\boldsymbol{w}}$ resembles the TER solution in previous lecture.

Consequently, the quadratic approximation to the AUC based objective and TAUC extension can be treated respectively as a Gaussian process taking pairwise data $(\boldsymbol{x}_j^- - \boldsymbol{x}_i^+)$ as regression input and a corresponding scaled version.

Summary of Classifier Relationships

Table 1: Summary of classifiers and settings

		class-size, reg, offset				scaling				translation			
	Classifier	m^+	m^-	b	η	β_1	β_2	γ_1	γ_2	$\boldsymbol{u}_1$	$\boldsymbol{u}_2$	$\boldsymbol{v}_1$	$\boldsymbol{v}_2$
1.	FLD	1	1	b	1/2	1	0	0	1	$\boldsymbol{\mu}_+$	$\mathbf{0}$	$\mathbf{0}$	$\boldsymbol{\mu}_-$
2.	LSE	1	1	b	1/2	1	0	0	1	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$
3.	TER	m^+	m^-	b	1/2	1	0	0	1	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$
4.	AUC	m^+	m^-	b	1/2	1	1	1	1	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$	$\mathbf{0}$
5.	Novel	m^+	m^-	b	1/2	r	r	r	r	r	r	r	r

m^+, m^- : sizes of positive class and negative class training samples respectively,
b : weight-decay regularization parameter which is chosen as 10^{-4},
η, r : η is quadratic offset as seen in (5.103), r is random number,
$\boldsymbol{\mu}_+, \boldsymbol{\mu}_-$: $\boldsymbol{\mu}_+ = \frac{1}{m^+}\sum_{j=1}^{m^+} \boldsymbol{p}_i^+$ which is mean of positive class features,
$\boldsymbol{\mu}_- = \frac{1}{m^-}\sum_{j=1}^{m^-} \boldsymbol{p}_j^-$ which is mean of negative class features,
$\beta_1, \beta_2, \gamma_1, \gamma_2$: scaling parameters (scalars),
$\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{v}_1, \boldsymbol{v}_1$: translational parameters (vectors).

Thank you!

Duda, R. O., Hart, P. E., and Stork, D. G. (2001).
Pattern Classification.
John Wiley & Sons, Inc, New York, 2nd edition.